

TERESA — Braccio Unitree Z1

Diario di sviluppo, TODO list e guida di avvio

Branch attivo: `Z1_realsense` | Repo: `TERESA_ws/src/z1_vision`

Ultimo aggiornamento: **11 Aprile 2026**

1 Panoramica del sistema

Il sistema controlla il braccio **Unitree Z1** montato sul robot TERESA per avvicinarsi in modo autonomo al torso di un paziente. Lo stack completo è composto da:

- **Percezione:** YOLO11 pose estimation + Kalman 3D + lock tracker + body search scan
- **Pianificazione:** Workspace checker (Pinocchio + L-BFGS-B)
- **Controllo posizione:** IK Jacobiana pseudo-inversa smorzata + JointTrajectoryController
- **Controllo forza:** Impedance controller in spazio cartesiano (500 Hz, torque control)
- **Sicurezza:** keyboard safety node, stati HOMING/EMERGENCY, workspace clipping
- **Orchestrazione:** FSM a 12 stati, launch files separati per controllo e percezione
- **Integrazione Spot:** package `spot_perception` su Jetson (Orbbec + YOLO skeleton + classificazione postura)

2 Branch attivi

Branch `Z1_realsense` — branch principale attivo (2026-04-11)

Contiene l'intero stack: FAST protocol, body search scan 3 fasi, impedance controller, `spot_perception` package. `FSM_main` è la base stabile precedente (2026-03-15), da cui questo branch è stato esteso.

3 Architettura FSM (stato attuale — Z1_realsense)

Stati della FSM (z1_FSM.py)

Stato	Descrizione
HOMING	Stato iniziale al boot. Porta il braccio in home pose via IK/JTC, poi WAITING
WAITING	Se <code>body_scan_on_start=true</code> e scan non ancora fatto → BODY_SCANNING; altrimenti aspetta target LOCKED
BODY_SCANNING	Scansione a griglia per trovare la posizione ottimale di visione del torso
CHECKING_WORKSPACE	Verifica raggiungibilità target (thread non bloccante, Pinocchio)
APPROACHING	Invia goal IK verso il target (latchato al momento del check)
WAIT_IK_DONE	Attende <code>/ik_done=True</code> dal nodo IK
WRIST_ALIGN	Riallinea il polso perpendicolare alla superficie torso
SWITCHING_TO_TORQUE	Switch JTC → torque_controller via <code>safe_controller_switch</code>
IMPEDANCE_RUNNING	Impedance controller attivo: approach → hold → retract
SWITCHING_TO_JTC	Switch torque_controller → JTC
EMERGENCY_SWITCHING	Forza switch a JTC se torque attivo, poi va in HOMING
EMERGENCY	Stato bloccante, aspetta solo comando <code>reset</code> da tastiera
FAULT	Errore fatale (switch fallito), nodo bloccato

Topic principali (inclusi i nuovi di Z1_realsense)

Topic	Tipo	Descrizione
<code>/torso_target_ee_locked</code>	PoseStamped	Target EE quando tracker LOCKED
<code>/torso_target_ee</code>	PoseStamped	Target EE interpolato continuo
<code>/ik_goal_pose</code>	PoseStamped	Goal IK da FSM
<code>/ik_enable</code>	Bool	Abilita nodo IK
<code>/ik_done</code>	Bool	Conferma fine movimento JTC
<code>/impedance_enable</code>	Bool	Abilita impedance controller
<code>/impedance_done</code>	Bool	Fine ciclo impedance
<code>/target_out_of_workspace</code>	Bool	Target clippato dal workspace checker
<code>/z1_fsm/state</code>	String	Stato corrente FSM
<code>/z1_keyboard_cmd</code>	String	Comandi da tastiera
<code>/torso_surface_frame</code>	PoseStamped	Piano superficie torso (PCA)
<code>/surface_signed_distance</code>	Float32	Distanza TCP-superficie
<code>/tracker_scan_mode</code>	Bool	Attiva/disattiva scan mode nel tracker
<code>/tracker_scan_next</code>	Bool	Avanza al punto successivo della griglia
<code>/torso_scan_point</code>	Float32MultiArray	Score/dati del frame corrente [score, n_kp, conf, x, y, z]

4 DONE — Funzionalità implementate

Sessione 2026-03-15 (branch FSM_main)

- ✓ Pipeline base: YOLO11 + Kalman 3D + FSM IDLE/ESTIMATING/LOCKED
- ✓ IK Jacobiana pseudo-inversa smorzata + smoothstep quintico + JTC
- ✓ `realsense_surface_node`: stima piano torso da depth ROI (PCA)
- ✓ Impedance controller (500 Hz): APPROACH → HOLD → RETRACT
- ✓ `safe_controller_switch`: switch JTC ↔ torque come nodo persistente
- ✓ Workspace checker (Pinocchio + L-BFGS-B), clip target a $\text{max_reach} - 30$ cm
- ✓ HOMING, EMERGENCY_SWITCHING, EMERGENCY, keyboard safety (H/ESC/-R/Q)
- ✓ Launch files: `z1_control`, `z1_perception`, `z1_realsense`
- ✓ **Fix orientamento EE**: `ik_rot_weight` portato da 0.5 a 0.8 per garantire EE perpendicolare alla superficie durante il JTC approach

Sessioni 2026-03-20 → 2026-03-31 (branch Z1_realsense)**Body Search Scan 3 fasi — `body_search_scanner.py`**

- ✓ Classe `BodySearchScanner`: FSM interna 3 fasi (wrist sweep in home → arco nel piano $X \times Y$ a quota fissa → refinement adattivo)
- ✓ Score per punto: $\text{detection_rate} \times \text{avg_per_frame_score}$, con per-keypoint confidence tracking e completeness-weighted fusion
- ✓ Stima 3D torso per triangolazione multi-vista con anchor-based outlier rejection
- ✓ Early stop: se score $\geq$ soglia interrompe la scansione anticipatamente; fallback al miglior punto visto se nessuno supera la soglia
- ✓ Fase 3 fallback: se FAILED usa la stima di fase 2 invece di abortire
- ✓ Fase 3 pubblica sempre il seed al tracker anche se fallisce

Protocollo FAST — `z1_scan_manager.py` + `z1_FSM.py`

- ✓ Protocollo FAST implementato con punti anatomici derivati dai keypoint YOLO: Centro hub → Sottotifoidea (cardiaca) → RUQ Morrison's → LUQ Koller's → Sovrapubica
- ✓ Ridotto a **4 punti torso-only** (rimossa Pleurica Dx, troppo craniale): tutti i punti clippati a $Y \in [\text{shoulder_mid_y}, \text{hip_mid_y}]$ prima del calcolo dell'offset — il braccio non esce mai dal bounding box del torso
- ✓ `skip_impedance` esegue il dry-run completo del FAST via SCAN_PRELIFT: centro hub → SCAN_PAUSE → pt1 → PRELIFT → centro → ... → pt4 → WAITING
- ✓ **Fix**: `advance()` su center hub chiamato *prima* di SCAN_PAUSE per evitare loop infinito hub → SCAN_PAUSE → hub

Fix orientamento e stabilità

- ✓ **Fix:** `_orientation_for_xee` ripristinata a riferimento `Y_home` (Gram-Schmidt): un commit precedente aveva silenziato questa modifica usando `Z_world`, causando rotazione polso di 90 attorno all'asse di approccio
- ✓ **Fix:** rimossa doppia dichiarazione `scan_pause_s` in `__init__` della FSM
- ✓ Stima forza di contatto model-based (gravity + coriolis) senza sensore di forza esterno
- ✓ Look-at dinamico in fase 2: l'EE punta verso il torso stimato ad ogni posa dell'arco

Sessione 2026-04-11 — Integrazione Spot + pulizia workspace**Nuovo package `spot_perception` in `TERESA_ws`**

- ✓ Creato package `src/spot_perception` (ament_python) per girare sulla Jetson
- ✓ Migrati da `ros2_spot_ws`: `yolo_skeleton_spot.py`, `posture_classifier.py`, `laying_human_detector.py`, `depth_processing.py`, `skeleton_utils.py`, `kalman_filter.py`, `visualization.py`
- ✓ Launch file `spot_perception.launch.py` con parametri GPU Jetson (`device: '0'`, `use_half: true`)
- ✓ Driver Orbbec (`src/orbbec_camera`) già presente in `TERESA_ws`

Pulizia workspace

- ✓ Rimossi package obsoleti: `z1_control` e `yolo_pose_ros` (superseded da `z1_vision` e `spot_perception`)

Decisione architetturale Jetson / SpotCore

- ✓ **Jetson** (Orbbec USB + RealSense USB + Z1 USB): `spot_perception` + `z1_vision` + driver camere + driver Z1
- ✓ **SpotCore**: solo `spot_ros2` (driver hardware Spot) + nodi navigazione (`target_tracker`, `cmd_vel_controller`, `odom_nav`)
- ✓ Comunicazione Jetson ↔ SpotCore via ROS2 DDS (stesso `ROS_DOMAIN_ID`)
- ✓ Installazione nativa su Jetson (no Docker): motivazione latenza real-time 500 Hz + accesso diretto USB + CUDA nativo L4T

5 TODO**Alta priorità — Jetson setup & integrazione**

- ☐ **Jetson setup (installazione nativa):** ROS2 Humble, Pinocchio, driver Unitree Z1, RealSense SDK, Orbbec SDK, Ultralytics YOLO11
- ☐ **Test `spot_perception` su Jetson:** verificare che `yolo_skeleton`, `posture_classifier`

e `laying_human_detector` girino correttamente con Orbbec reale

- ☐ **Nodo orchestratore:** coordinare “Spot arrivato in posizione” → trigger avvio FAST scan su Z1 (topic/servizio tra SpotCore e Jetson via DDS)
- ☐ **Configurare ROS_DOMAIN_ID** comune tra Jetson e SpotCore; verificare che i topic attraversino la rete correttamente

Alta priorità — test su robot reale (Z1)

- ☐ **Testare il ciclo FAST completo:** HOMING → BODY_SCANNING → WAITING → CHECKING_WORKSPACE → APPROACHING → IMPEDANCE → SCAN_PRELIFT → loop 4 punti
- ☐ **Calibrare** `body_scan_center` e `body_scan_lookat_distance` sul robot reale
- ☐ **Verificare Y clipping FAST:** controllare che tutti i punti rimangano entro il bounding box del torso nelle varie posture paziente
- ☐ **Verificare l’orientamento look-at:** controllare nei log che l’EE ruoti correttamente spostandosi in alto/basso e lateralmente

Media priorità

- ☐ **Unificare pipeline YOLO sulla Jetson:** singola istanza modello YOLO11 per Orbbec (skeleton 17 joint) e RealSense (torso center), evitando doppio caricamento pesi in VRAM
- ☐ **Test comunicazione Jetson ↔ SpotCore:** verificare latenza e affidabilità dei topic ROS2 critici (`/laying_human/approach_point`, trigger orchestratore)
- ☐ **Merge Z1_realsense su FSM_main** dopo validazione sul robot reale
- ☐ **Tuning griglia body scan:** valutare se `ext_y` e `ext_z` coprono correttamente la zona torso sul robot reale
- ☐ **Tuning** `body_scan_min_frames`: verificare che 10 frame siano sufficienti per una stima affidabile dello score

Bassa priorità / futuri miglioramenti

- ☐ **Usare** `/target_out_of_workspace`: comandare a Spot di avvicinarsi quando il target è fuori workspace Z1
- ☐ **Logging:** ROS bag recording automatico al launch con i topic principali (FSM state, torso target, scan point, impedance)
- ☐ **Visualizzazione RViz:** marker per griglia scan, look-at target, piano superficie, workspace reachability, skeleton Spot
- ☐ **Multi-persone:** gestione del caso in cui YOLO rileva più persone (attualmente prende la prima/più vicina)

6 Sequenza di avvio

Prerequisito: workspace ROS2 compilato e sourced.

```
cd ~/Documents/GIT_Repositories/TERESA_ws
colcon build -packages-select z1_vision && source install/setup.bash
```

Terminale 1 — Hardware robot + camera + TF

```
ros2 launch z1_vision z1_realsense.launch.py
```

Terminale 2 — Percezione (YOLO + surface)

```
ros2 launch z1_vision z1_perception.launch.py
```

Terminale 3 — Stack di controllo

```
# Avvia: safe_controller_switch + z1_ik_to_jtc + impedance (t=0s)
#       z1_FSM dopo 5s (primo stato: HOMING -> BODY_SCANNING -> ...)
ros2 launch z1_vision z1_control.launch.py
```

Terminale 4 — Keyboard safety

```
ros2 run z1_vision z1_keyboard_safety
```

Tasto	Azione
H	Invia braccio alla home position (qualunque stato)
ESC	Emergenza: stop immediato, poi HOMING
R	Reset dallo stato EMERGENCY
Q	Chiude il nodo keyboard safety

Monitoraggio

```
ros2 topic echo /z1_fsm/state           # stato FSM
ros2 topic echo /torso_target_ee_locked # target YOLO
ros2 topic echo /surface_signed_distance
ros2 topic echo /torso_scan_point       # score scan (durante
BODY_SCANNING)
ros2 run z1_vision fk_reader             # FK end-effector
```

7 File principali

File	Ruolo
z1_FSM.py	FSM principale (12 stati, include BODY_SCANNING)

File	Ruolo
<code>body_search_scanner.py</code>	Classe <code>BodySearchScanner</code> : FSM interna griglia scan
<code>z1_ik_to_jtc.py</code>	IK Pinocchio → JTC action
<code>impedance_controller_realsense.py</code>	Controllo impedenza cartesiana 500 Hz
<code>safe_controller_switch.py</code>	Switch JTC ↔ torque (servizi ROS2)
<code>workspace_checker.py</code>	Raggiungibilità workspace (Pinocchio + scipy)
<code>z1_yolo_torso_tracker.py</code>	YOLO11 + Kalman 3D + lock + scan mode
<code>realsense_surface_node.py</code>	Stima piano torso (PCA depth ROI)
<code>z1_keyboard_safety.py</code>	Safety node interattivo da terminale
<code>kalman_filter.py</code>	Filtro Kalman 3D [x,y,z,vx,vy,vz]
<code>fk_reader.py</code>	Utility: FK end-effector da <code>joint_states</code>
<i>Package <code>spot_perception</code> (Jetson, Orbbec)</i>	
<code>yolo_skeleton_spot.py</code>	YOLO11 skeleton 17 joint + Kalman + TF → body frame
<code>posture_classifier.py</code>	Classifica STANDING / SITTING / LYING
<code>laying_human_detector.py</code>	Rileva persona sdraiata → calcola approach point
<code>depth_processing.py</code>	Depth lookup con window mediana + outlier filter
<code>skeleton_utils.py</code>	Constraint lunghezza torso, COCO edges, smoothing
<code>config/z1_fsm_params.yaml</code>	Parametri FSM (home pose, topic, workspace)
<code>config/z1_ik_jtc_params.yaml</code>	Parametri IK/JTC (<code>ik_rot_weight: 0.8</code>)
<code>config/z1_yolo_torso_params.yaml</code>	Parametri YOLO/Kalman/lock
<code>config/impedance_control_params.yaml</code>	Parametri impedance (guadagni, timing)
<code>config/surface_params.yaml</code>	Parametri surface node
<code>config/body_search_params.yaml</code>	Parametri griglia scan (<code>ny×nz</code> , <code>lookat</code> , <code>timeout</code> , <code>early stop</code>)
<code>launch/z1_control.launch.py</code>	Avvia stack controllo
<code>launch/z1_perception.launch.py</code>	Avvia stack percezione
<code>launch/z1_realsense.launch.py</code>	Avvia HW robot + camera + TF

8 Note tecniche

- **IK**: Jacobiana pseudo-inversa smorzata (*damped least squares*), frame LOCAL, smoothstep quintico (zero velocità a inizio/fine), joint unwrap con `_make_target_near`. `ik_rot_weight: 0.8` per EE perpendicolare alla superficie
- **Kalman**: dt adattivo dal timestamp immagine, `vel_damping = 0.9`, covarianza noise configurabile da YAML
- **Lock tracker**: varianza soglia su 20 frame + 5 check consecutivi sotto soglia
- **Body scan look-at**: `lookat = home_pos + lookat_dist × X_ee_home`. Rotazione per ogni punto griglia via `_orientation_for_xee()` (Rodrigues, rotazione minima da home). Errore angolare < 5 per variazioni di distanza ±50%

- **Impedance:** 500 Hz, PID cartesiano + joint-space polso (J4–J6), compensazione gravity+coriolis, Jacobiano LOCAL_WORLD_ALIGNED
- **URDF path** (hardcoded):
/home/andrea/Ros2_repositories/unitree_z1_ws/install/
z1_description/share/z1_description/urdf/z1.urdf
- **Branch:** FSM_main (stabile) ← base di Z1_realsense (feature body scan, da testare sul robot reale poi fare merge)